
signus

개발문서 — 구조 · 확장 · 로드맵

블라인드 PSK/QAM/FSK 복조기 + 광대역 다중신호 Survey 프론트엔드. 이 문서는 코드베이스에 처음 들어온 개발자가 구조를 파악하고, 안전하게 고치고, 확장할 수 있도록 모듈 지도·데이터 흐름·테스트 방법론·성능 계층·로드맵을 정리합니다.

선행 문서:

signus 원리 이해 설명서

(DSP 개념·각 단계의 원리)

브랜치:

`wideband-frontend`

· 미병합(master 대상) · 2026-07

런타임 의존성: numpy + scipy (numba는 선택적

`fast`

extra)

목차

- 1. 개요와 하드 제약
- 2. 저장소 구조 (모듈 지도)
- 3. 실행 방법 — 설치 · CLI · 웹 · 테스트
- 4. 핵심 데이터 흐름 (analyze · survey)
- 5. 주요 데이터 구조
- 6. 테스트 · 캘리브레이션 방법론
- 7. 성능 최적화 계층 (numba + 순수 numpy) · **numba 결정**
- 8. 확장 가이드
- 9. 코딩 규약 · 함정
- 10. 로드맵 — 다음 스텝 · 버그 · 한계

1. 개요와 하드 제약

드래그&드롭한 PCM/IQ 파일에서 신호 제원을 **블라인드**로 자동 탐지·복조하고, 시간순 애니메이션 컨스텔레이션과 락 점수를 보여준다. 원리는 “원리 이해 설명서” 참조. 아래 제약은 **모든 변경이 반드시 지켜야 하는 것이다**.

제약	실무적 의미
런타임 의존성 = numpy + scipy 뿐	새 런타임 import 추가 금지. numba는 선택적 이며 없으면 동일 동작으로 폴백.
Air-gap · 손타이핑 · 줄 예산	망분리 기계에 사람이 직접 타이핑. 외부 통신 0(UI는 stdlib http.server). 코드 줄 수도 비용이다.
BER=0 캘리브레이션	CORE 53케이스 비트오류 0을 상시 유지. 수치를 바꾸는 변경은 전수 스윕으로 증명해야 한다.
Anti-shared-bug	gen.py (생성기)와 수신기 DSP는 코드 공유 금지(상수표만). 예: TX/RX RRC를 합치면 위반.
파일명은 정답이 아님	파일명엔 fs/fmt/dtype만. 진짜 정답은 <file>.json 사이드카이며 분석기는 절대 읽지 않는다.

2. 저장소 구조 (모듈 지도)

모듈	역할	핵심 심볼
constellations.py	변조 상수의 단일 출처(이상점·Gray·차동표·FSK레벨)	ideal_points, bit_labels, MODS
sigio.py	샘플 I/O, 메타, SigMF·파일명 파싱	read, write, Meta, parse_name
gen.py	합성 신호 생성기(테스트 정답 소스). 수신기와 코드 공유 안 함	generate, GenParams, save
dsp.py	수신 DSP 단계 모음(반송파·baud·정합필터·타이밍·미세동기). 공유 _kmeans1d / _parab	est_carrier, est_baud, matched, timing, ddsync
classify.py	변조 분류·회전 정렬·품질·락·SNR	classify, align, quality, snr_m2m4
eq.py	블라인드 등화기 CMA→DD (심볼간격 + T/2 FSE)	equalize, equalize_fse

fsk.py	FSK/MSK 경로: 포락선 게이트 + 주파수 판별기	fsk_gate, analyze_fsk
_accel.py 신규	선택적 numba 커널(순차 루프). 없으면 동일 코드가 순수 numpy 로	cma_dd_equalize(_fse), dd_carrier, HAVE_NUMBA
detect.py wideband	광대역 탐지: STFT→CFAR→라벨→에미터 상자	detect, Detection
channelize.py wideband	에미터 하나를 기저대역 채널로(mix+LPF+decimate)	extract
trriage.py wideband	채널을 linear/fsk/analog/tone으로 정직 분류	family
pipeline.py	오케스트레이션: <code>analyze()</code> (단일) · <code>survey()</code> (다중). Result/Emitter/Survey	analyze, survey, Result
spectrum.py	스펙트럼·폭포수 뷰(UI 표시용 + 탐지 입력)	spectrum, waterfall
server.py	stdlib http.server 기반 웹 UI 백엔드	run
cli.py	CLI: analyze·survey·gen·dataset·sweep·serve	main
web/	정적 UI(index.html·app.js·style.css). Canvas 2D 컨스텔레이션	—

임포트 방향(비순환): `pipeline` → `{classify, dsp, fsk, eq, detect, channelize, triage}` → `{dsp, _accel, constellations}`. `gen` 은 수신 경로에서 격리.

3. 실행 방법

```
# 설치 (개발)
pip install -e ".[dev]"          # numpy scipy pytest ruff
pip install -e ".[dev,fast]"     # + numba (서버/대용량용; numpy를 <2.5로 내림)

# 단일 신호 분석
python -m signus.cli analyze capture.iq --fs 1e6
# 광대역 다중신호 조사
python -m signus.cli survey wideband.iq --fs 20e6 --report out.json
# 합성 신호 생성 (정답 사이드카 포함)
python -m signus.cli gen --mod 16qam --snr 20 --out samples/
# 전수 스윕(회귀 검증): 그리드 생성→복조→채점
python -m signus.cli sweep --tier core   # CORE 53케이스 BER=0
python -m signus.cli sweep --tier full   # + STRETCH
# 웹 UI
python -m signus.cli serve --port 8000

# 테스트 · 린트
pytest -q                          # 247 통과
ruff check signus/ tests/
SIGNUS_NO_NUMBA=1 pytest -q       # numba 설치돼 있어도 폴백 경로 강제 검증
```

4. 핵심 데이터 흐름

4.1 analyze() — 단일 신호

```
analyze(x, meta):
    x = analytic(x); x -= mean          # 프론트엔드 + DC 블록
    bursts = find_bursts(x); pick 최강  # 버스트 선택
    if fsk_gate(xb): return analyze_fsk(xb)    # — FSK 분기
    fc = resolve_alias(est_carrier(xb))      # 반송파 + 에일리어스 해소
    xd = mix(xb, fc)
    baud, conf = est_baud(xd)
    alpha, ym, raw, mod, syms, q = _demod(xd, baud)    # 정합필터→타이밍→분류→미세동기→품질
    if occupied_bw 불일치: in-band baud 후보 재시도 (락으로 경쟁)
    if q.lock < 60:                          # — 등화기 레스큐
        _rescue(equalize) ; 안되면 _rescue(equalize_fse)
        락이 충분히 오를 때만 채택
    bits = demap_bits(syms, mod)
    return Result(...)
```

`_demod` 내부에서 `ddsync` (미세동기)와 `align` 이, `_rescue` 에서 `equalize/equalize_fse` 가 호출된다 — 이들이 성능 핫스팟(7장).

4.2 survey() — 광대역 다중신호

```
survey(x, meta):
    xa = analytic(x) - mean
    for d in detect(xa, fs):              # 에미터마다
        ch, fs_ch = extract(xa, fs, d)    # 채널 잘라내기
        kind = triage.family(ch, fs_ch)    # 디지털? 아날로그? 톤?
        if kind in (linear, fsk):
            try: r = analyze(ch, Meta(fs_ch,...))    # 채널별 격리(try/except)
            except: kind = 'error'                # 한 채널 실패가 전체를 막지 않음
        emitters.append(Emitter(d, kind, abs_fc, r))
    return Survey(meta, emitters)
```

5. 주요 데이터 구조

구조	필드(발체)	용도
Meta	fs, fmt, dtype, endian, bitrev	샘플 해석 규격
GenParams	mod, baud, fc, snr, taps, drift_ppm...	생성기 입력(테스트 정답의 근원)
Result	family, fc, baud, mod, lock, symbols, bits, evm, mer_db, eq_applied...	단일 분석 결과 + <code>to_json</code> (UI)
Detection	fc, bw, t0, t1, snr_db, baud_hint	탐지된 에미터 상자
Emitter / Survey	detection, kind, abs_fc, result / emitters[]	광대역 조사 결과

6. 테스트 · 캘리브레이션 방법론

- **정답의 근원은 gen.py.** 생성기가 만든 신호+ `.json` 사이드카가 유일한 ground truth. 기대값을 수신기 알고리즘으로 재유도하면 anti-shared-bug 위반이다.
- **sweep이 회귀 게이트:** `--tier core` = 53케이스 BER=0(반드시 유지), `--tier full` = +STRETCH(현재 10/11, bpsk/lowsnr 1건은 알려진 한계).
- **단위 테스트 247개(pytest).** 수치를 건드리는 변경은 **pytest + CORE 스왑 둘 다** 그린이어야 한다.
- **캘리브레이션 상수(예: C42 게이트 0.80, 링 비율, 락 스케일)**는 실제 프론트엔드를 통과시킨 스왑으로 맞춰졌다. 함부로 바꾸지 말 것.

7. 성능 최적화 계층

프로파일 결과 병목은 두 종류로 갈린다. 각각 다른 해법을 쓴다.

7.1 종류 A — 벡터화됐지만 낭비 (순수 numpy로 해결, 모두에게 적용)

대상	변경	효과
<code>classify._nearest_dist</code>	(...,M) 브로드캐스트 → pts에 대한 running <code>np.minimum</code>	비트동일 1.5-5.5×
<code>dsp._kmeans1d</code>	classify-fsk 중복 통합 + running-argmin(strict < 가 최저인덱스 tie 보존)	비트동일 1.5-2.2×

두 변경은 라벨/스프레드가 브로드캐스트 argmin과 완전히 동일함을 수치로 검증했다(0 불일치). numba 없이도 적용된다. `_parab` ·argparse 중복 제거도 함께 수행.

7.2 종류 B — 진짜 순차 피드백 루프 (numba로만 가속)

`eq.equalize_fse` (118ms) · `eq.equalize` (38ms) · `dsp.ddsync` (18ms)는 매 갱신이 직전 출력에 의존해 **벡터화 불가**. `_accel.py` 가 이 루프를 **기존과 동일한 식으로 @njit** 커널에 담는다.

```
# _accel.py 핵심 구조
try:    import numba; njit = numba.njit(cache=True, fastmath=False); HAVE_NUMBA=True
except ImportError: njit = lambda fn: fn; HAVE_NUMBA=False # 동일 코드가 순수 numpy로
# SIGNUS_NO_NUMBA=1 이면 설치돼 있어도 폴백 강제
```

지표	측정값
루프 단독(steady-state)	ddsync 34x · equalize 25-30x · equalize_fse 16x
무거운 16qam/2ray analyze(warm)	470ms → 390ms(순수numpy) → 177ms(+numba) . 전체 ~2.65×
검증	폴백=원본 byte-identical (0.0) · numba=원본 8.9e-16 · 양쪽 247테스트/CORE53/STRETCH10 동일

numba의 비용 ① import ~380ms + 최초 컴파일 ~280ms(cold)/~60ms(warm cache). 그래서 **신호 하나짜리 단발 CLI는 오히려 느려진다**(기동비용 > 절감). ② numba 0.66은 **numpy<2.5**를 강제 → 설치 시 numpy 2.5.1→2.4.6 다운그레이드(테스트로 안전 확인). ③ llvmlite(수십MB 바이너리)를 air-gap 기계에 올려야 함.

결정: numba를 쓴다 — 단 “조건부”로. 서버(**serve**)와 대용량 **survey** 처럼 한 프로세스에서 등화기/미세동기를 여러 번 호출하는 경로에서만 켜다(`pip install -e ".[fast]"`). 이때 무거운 신호가 2-2.65× 빨라지고 기동비용은 상각된다. 단발 CLI·개발·기본 설치는 numba 없이(numpy 2.5 유지) 두고, 순수 numpy 개선분만으로도 충분히 빠르며 결과는 **완전히 동일**하다. 코드는 이미 이 양면을 모두 지원한다(옵션 extra + 폴백 + 환경변수 탈출구).

8. 확장 가이드

새 변조 클래스 추가 (예: APSK, 128/256QAM)

- `constellations.py` : `_INFO` 에 (차수, 대칭성) 추가, `ideal_points` · `bit_labels` 확장.
- `classify.py` : 대칭성 4 그룹이면 새 분리 지표(큐물런트/링) 추가 — **블라인드로 가를 수 있어야** 등재. 못 가르면 gen-only.
- `gen.py` : 생성 경로 추가(수신기 코드 재사용 금지).
- `cli.py sweep` 그리드에 케이스 추가 → CORE BER=0 확인.

새 탐지·분류 기능 (wideband)

`detect.py` (STFT/CFAR 파라미터), `triage.py` (비디지털 판정 특징) 확장. 채널 규약은 동결: `extract` 는 $bw \approx 0.28 \cdot fs_{ch}$ 로 맞춰 기존 엔진이 통하게 한다.

위임 정책 판단이 필요한 계층(통합·채점·수용 기준)은 리드가 직접 작성·리뷰하고 모든 산출물을 라인 단위로 감사한다. 서브에이전트는 “동결된 계약에 대한 기계적 작업”에만. 에이전트 산출은 반드시 **수치로 검증** 후 결합.

9. 코딩 규약 · 합정

- ruff 클린(E,F,I,UP,B,SIM,NPY), line-length 100. docstring 1-3줄.
- 수신 DSP는 constellations 외에 `gen` 을 절대 import하지 않는다.
- 순차 루프를 “벡터화”하려는 유혹 주의 — 등화기/ddsync는 의도적으로 순차. 커널화가 정답.
- 알려진 함정: `ber_bits` 는 심볼 단위 시프트 필요 · M2M4 SNR은 커토시스 보정 · pi4dqpsk는 qpsk로 도착(차동 디맵으로 비트 정확) · 반송파 에일리어스는 항상 해소.

10. 로드맵 — 다음 스텝 · 버그 · 한계

10.0 지금 병합 · 결정

- 결정** numba = 조건부 채택(7.2 참조). 기본 설치는 numpy 2.5 + 폴백, 서버/survey 배포만 `[fast]` .
- 병합** `wideband-frontend` 브랜치를 master로 병합(현재 미병합). 이번 최적화 커밋 포함.

10.1 단기 — 정리 · 알려진 버그/한계

항목	상태 · 조치
STRETCH bpsk/lowsnr 1건 실패	알려짐 저SNR BPSK 락 실패. 데이터-보조 추정(10.2-④)로 개선 여지.
pre-echo 채널 (<code>0.45e^-j0.5, 1, 0.3</code>)	범위 밖 baud+classify+eq를 동시 파괴(후보 정크가 진짜 rate를 락으로도 이김). 문서화된 한계.

16QAM + 강한 2ray	범위 밖 4제곱 반송파 톤 자체가 죽음.
oqpsk / pi4dqpsk	블라인드 클래스 아님(문서화). oqpsk는 gen-only, pi4dqpsk는 qpsk로 도착.
성능 대안: Pythran	후보 numpy pin·런타임 의존성 없이 eq 루프 컴파일(자립형 .so). 빌드 스텝/플랫폼 바이너리 필요 → 서버 배포에서 numba 대신 검토 가치.

10.2 중기 — 미착수 서브프로젝트 (wideband 로드맵)

#	서브프로젝트	내용 · 이유
3	대용량 스트리밍 I/O	GB급 캡처를 청크/mmap로 읽고 읽으면서 decimate. 현재 전체를 RAM에 적재(1GB ci16→~13GB). 서버 _MAX_BODY 상향.
4	짧은 버스트 · 데이터-보조 추정	AIS 슬롯 ~232심볼. 블라인드 4제곱은 짧은 버스트에서 무너짐 → 프리앰블 상관(Fitz/L&R)로 13-180× 개선, 0dB까지. 순서도 baud→timing→carrier로.
5	프레이밍/디코드 계층	AIS(NRZI+HDLC+비트스터핑+CRC-16)·DSC·NAVTEX 메시지 복원. 현재 프레이밍 계층 없음.
6	웹 UI 폭포수 + 상자	탐지 상자 오버레이 폭포수 → 클릭하면 해당 에미터 분석으로 드릴다운.
7	RF 절대 중심주파수	Meta.rf_center 추가 → 기저대역 fc를 실제 RF 주파수로 보고.

10.3 장기

- SDR 플러그인 · 실시간 스트림, 페이딩 + 추적 등화기.
- 등화기 before/after 토글, 폴더 감시, HTML 리포트.
- 비디지털 정밀 판정(레이더 PRF 콤, DSSS의 BW·T 플래그) 강화.

10.4 권장 진행 순서

1. 이번 최적화 + wideband를 **master 병합** (게이트: 247 + CORE53 + STRETCH10 + ruff).
2. **#6 UI 폭포수-상자** — survey가 이미 데이터를 주므로 가시성 효과 대비 저비용, 사용성 즉시 향상.
3. **#3 스트리밍 I/O** — 실전 GB 캡처의 선결 조건.
4. **#4 짧은-버스트 DA** → **#5 AIS 프레이밍** — 해상관제 실호 가치(메시지까지).
5. 서버 배포 성능이 문제가 되면 numba **[fast]** 켜기 또는 Pythran 검토.